

FATEC POMPEIA

Introdução à Computação em Nuvem · ISW055 · 161_SIST. INTELIGENTES_N

P1

Relatório bimestral de atividades

Avaliação individual · 2026.2

Guilherme Dias dos Santos

Prof. Allan Lincoln Rodrigues Siriani
Pompeia, 07/10/2026

Sumário

1	Introdução	2
2	Metodologia	2
3	Quadro de entregas	2
4	Atividades realizadas	3
4.1	Atividade 1 — Agenda telefônica em Flask	3
4.2	Atividade 2 — Catálogo de filmes — Tom Hanks	4
4.3	Atividade 3 — Desacoplando o login — microsserviço de autenticação	5
4.4	Atividade 4 — Controle de acesso por papel — RBAC	6
4.5	Atividade 5 — Logs e auditoria	7
4.6	Atividade 6 — Upload e perfil de usuário	9
5	Atividades extras (opcional)	10
5.1	Atividade E1 — Documentação Swagger/OpenAPI	10
6	Considerações finais	11
7	Declaração de autoria	12

1 Introdução

A disciplina Introdução à Computação em Nuvem (ISW055) tem como objetivo apresentar, de forma prática, conceitos relacionados ao desenvolvimento e à execução de aplicações em ambientes de computação em nuvem. Ao longo do semestre, as atividades são desenvolvidas progressivamente, partindo de uma aplicação monolítica e incorporando conceitos como persistência de dados, microsserviços, autenticação, controle de acesso, logs, armazenamento de arquivos e outras tecnologias utilizadas em arquiteturas modernas.

Durante o bimestre, o objetivo principal foi o desenvolvimento e a evolução de um catálogo de filmes baseado em Tom Hanks. A aplicação passou por diferentes etapas, incluindo o consumo da API do TMDb, persistência em banco de dados, criação de um microsserviço de autenticação, implementação de controle de acesso por papéis (RBAC), registro de logs e auditoria e, posteriormente, recursos de perfil, favoritos e armazenamento de imagens utilizando object storage.

Este relatório apresenta as atividades realizadas durante o bimestre, descrevendo a evolução do projeto e as principais tecnologias e conceitos aplicados em cada etapa. O documento está organizado de forma a apresentar inicialmente a metodologia utilizada, seguida pelo quadro de atividades e entregas, com as respectivas datas planejadas e realizadas, além das evidências de desenvolvimento disponíveis no GitHub.

2 Metodologia

Todas as atividades foram desenvolvidas no mesmo repositório público do GitHub, como continuação de um único projeto (catálogo de filmes). A data de cada entrega foi extraída de uma issue criada após a finalização da respectiva atividade, com uma menção ao professor para que ele fosse notificado. Os prints apresentados mostram as issues com a menção ao professor, o histórico de commits e o sistema em execução.

3 Quadro de entregas

Nº	Atividade	Descrição	Data planejada	Data realizada	Situação
1	Agenda telefônica em Flask	Nivelamento em sala: sistema monolítico Flask + Jinja com persistência em JSON.	07/08/2026	07/08/2026	entregue
2	Catálogo de filmes — Tom Hanks	Consumo da API TMDb, persistência em MariaDB e segregação por usuário.	20/08/2026	20/08/2026	entregue
3	Desacoplando o login — microsserviço de autenticação	Login, cadastro e esqueci-minha-senha num serviço à parte na rede interna do Docker.	28/08/2026	26/08/2026	entregue
4	Controle de acesso por papel — RBAC	O campo role passa a decidir permissões reais no backend (403 para usuário comum).	04/09/2026	03/09/2026	entregue

Nº	Atividade	Descrição	Data planejada	Data realizada	Situação
5	Logs e auditoria	Novo log-service com Redis registrando login, ações sensíveis e tentativas negadas.	25/09/2026	10/09/2026	entregue
6	Upload e perfil de usuário	Página de perfil com avatar no MinIO; só a referência fica no banco relacional.	02/10/2026	01/10/2026	entregue
E1	Documentação Swagger/OpenAPI	Swagger UI com pelo menos 2 serviços documentados e “Try it out” executado.	sem prazo	05/10/2026	entregue

4 Atividades realizadas

4.1 Atividade 1 — Agenda telefônica em Flask

Descrição	Nivelamento em sala: sistema monolítico Flask + Jinja com persistência em JSON.
Data planejada	07/08/2026
Data realizada	07/08/2026
Situação	entregue
Evidência	Realizada em sala — print do sistema rodando localmente e do repositório
Link	https://github.com/guilherme-dds/aula1

Nesta atividade foi desenvolvida uma aplicação simples de agenda telefônica utilizando Flask e Jinja2, com funcionalidades para cadastrar, listar e consultar contatos. Foi adotada uma estrutura monolítica, mantendo frontend e backend na mesma aplicação, e a persistência dos dados foi realizada por meio de um arquivo JSON, sem utilização de banco de dados ou containers. A atividade teve como objetivo compreender os conceitos básicos de desenvolvimento web com Flask antes da evolução do projeto para uma arquitetura voltada à computação em nuvem.

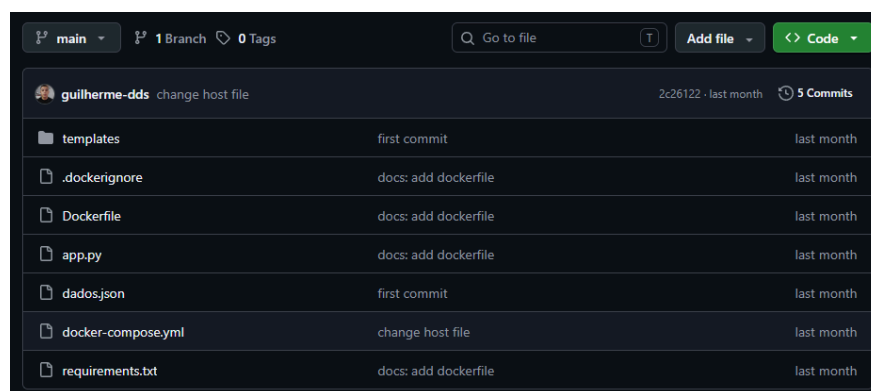


Figura 1: Atividade 1 — evidência da entrega

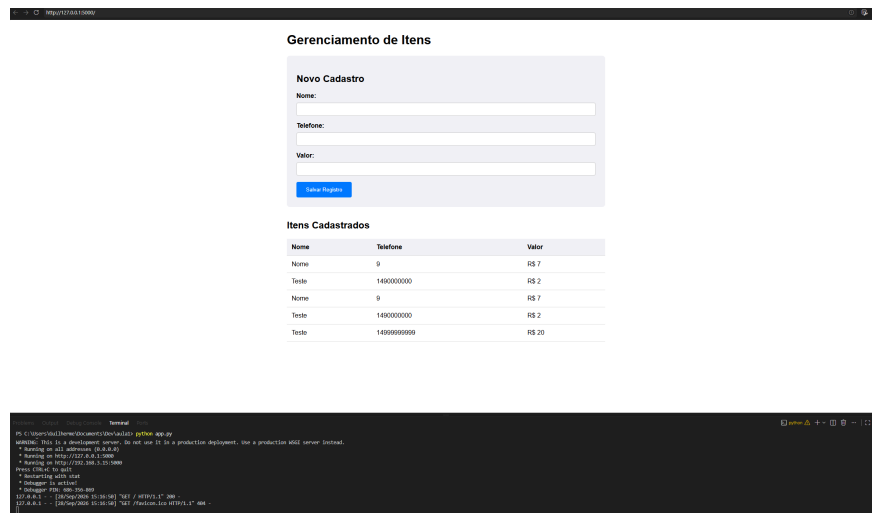


Figura 2: Atividade 1 — resultado (o sistema funcionando)

A principal dificuldade foi compreender a estrutura inicial do Flask e a comunicação entre rotas e templates. O problema foi resolvido organizando a aplicação de forma simples e utilizando o arquivo JSON como uma primeira forma de persistência.

4.2 Atividade 2 — Catálogo de filmes — Tom Hanks

Descrição	Consumo da API TMDb, persistência em MariaDB e segregação por usuário.
Data planejada	20/08/2026
Data realizada	20/08/2026
Situação	entregue
Evidência	GitHub — issue + print do catálogo
Link	https://github.com/guilherme-dds/movie-catalog

Nesta atividade a aplicação foi evoluída para um catálogo de filmes utilizando a API do TMDb, permitindo buscar e exibir informações sobre filmes relacionados ao ator Tom Hanks. Também foram implementados cadastro e login de usuários, favoritos e comentários, com os dados persistidos em um banco de dados relacional. Como decisão de arquitetura, as informações dos filmes continuaram sendo obtidas da API externa, enquanto o banco armazenava os dados relacionados aos usuários e suas interações. Docker também foi utilizado para facilitar a execução e publicação dos componentes da aplicação.

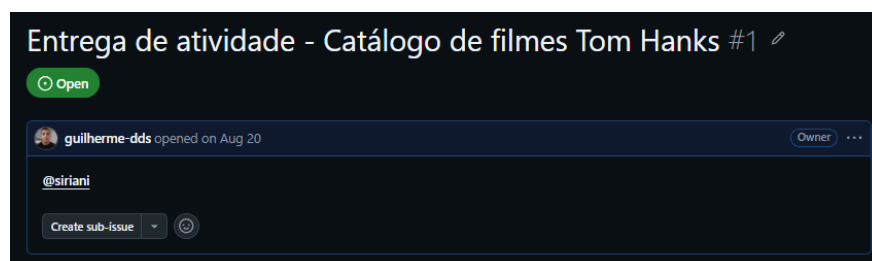


Figura 3: Atividade 2 — evidência da entrega

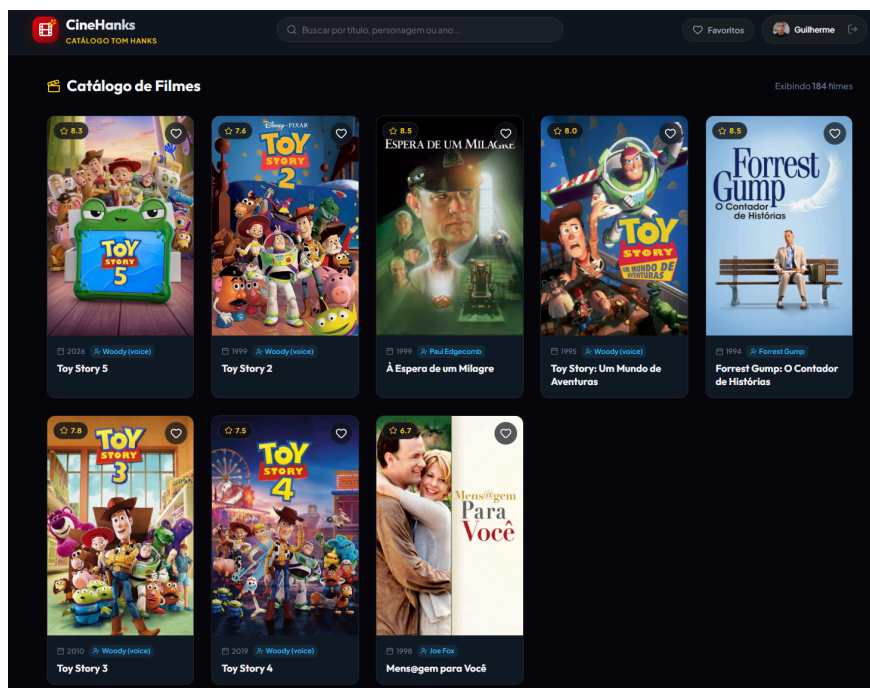


Figura 4: Atividade 2 — resultado

Um dos problemas encontrados foi a limitação de permissões do banco de dados disponibilizado para a disciplina, que impedia o Prisma de criar automaticamente o banco utilizado para migrations. Para contornar a limitação, foi utilizado o prisma db push, permitindo sincronizar o schema diretamente com o banco existente sem depender da criação de um banco auxiliar.

4.3 Atividade 3 — Desacoplando o login — microsserviço de autenticação

Descrição	Login, cadastro e esqueci-minha-senha num serviço à parte na rede interna do Docker.
Data planejada	28/08/2026
Data realizada	26/08/2026
Situação	entregue
Evidência	GitHub — issue + print do login funcionando
Link	https://github.com/guilherme-dds/movie-catalog

Nesta atividade a autenticação foi separada do backend principal do catálogo, dando origem a um microsserviço responsável pelo cadastro, login, controle de papéis e recuperação de senha. A comunicação entre os serviços passou a ser realizada por requisições HTTP, mantendo o serviço de autenticação independente do restante da aplicação. Também foram utilizados containers separados e a comunicação entre os serviços foi mantida pela rede interna do Docker, evitando a exposição desnecessária do microsserviço de autenticação.

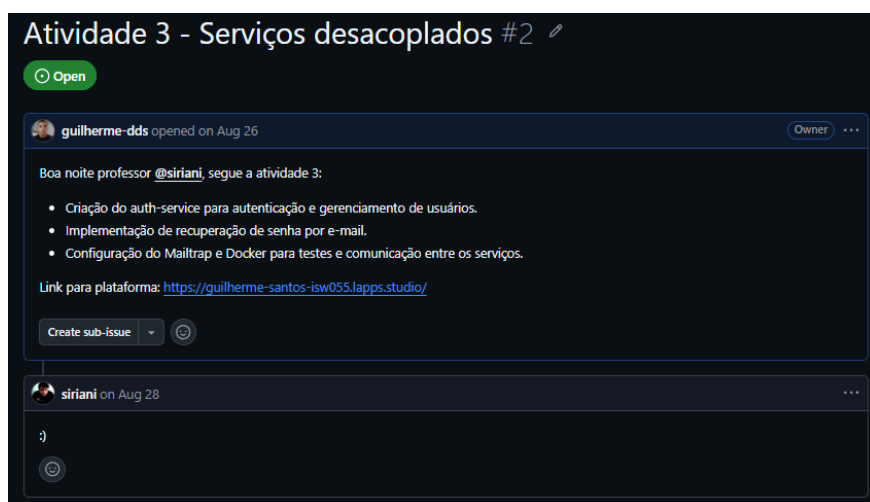


Figura 5: Atividade 3 — evidência da entrega

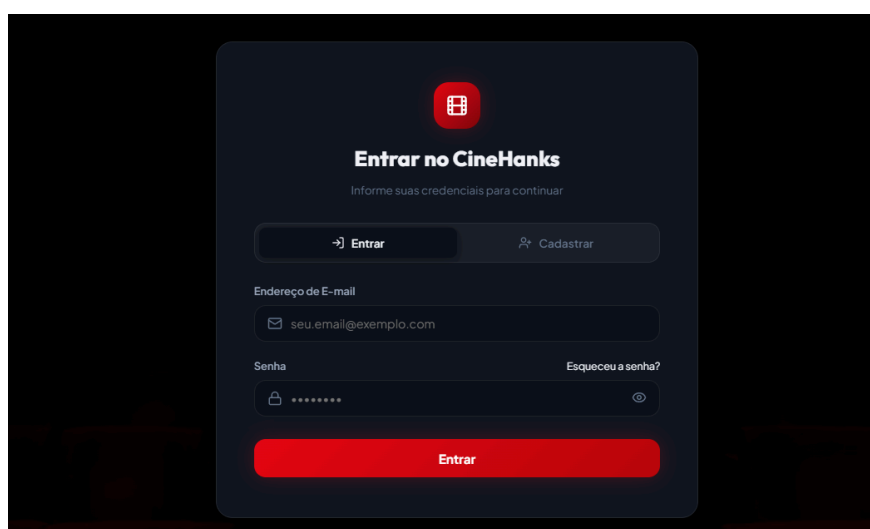


Figura 6: Atividade 3 — resultado

A separação da autenticação exigiu alterações nas rotas que anteriormente eram executadas diretamente pelo backend do catálogo. Foi necessário ajustar as requisições para que o backend passasse a consultar o auth-service e configurar os nomes dos containers na rede Docker para que os serviços conseguissem se comunicar sem utilizar localhost.

4.4 Atividade 4 — Controle de acesso por papel — RBAC

Descrição	O campo role passa a decidir permissões reais no backend (403 para usuário comum).
Data planejada	04/09/2026
Data realizada	03/09/2026
Situação	entregue
Evidência	GitHub — issue + funcionalidade disponível para ADMIN
Link	https://github.com/guilherme-dds/movie-catalog

Nesta atividade foi implementado o controle de acesso baseado em papéis, utilizando diferentes permissões para usuários comuns e administradores. O campo role passou a determinar quais funcionalidades

cada usuário poderia acessar, com as verificações sendo realizadas no backend. Como decisão de implementação, a autorização não ficou restrita ao frontend, garantindo que requisições feitas diretamente à API também fossem validadas e que usuários sem permissão recebessem uma resposta 403 Forbidden ao tentar acessar funcionalidades administrativas.

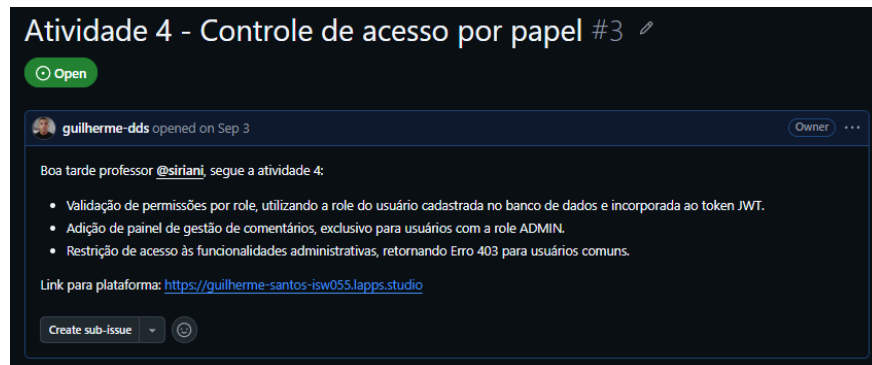


Figura 7: Atividade 4 — evidência da entrega

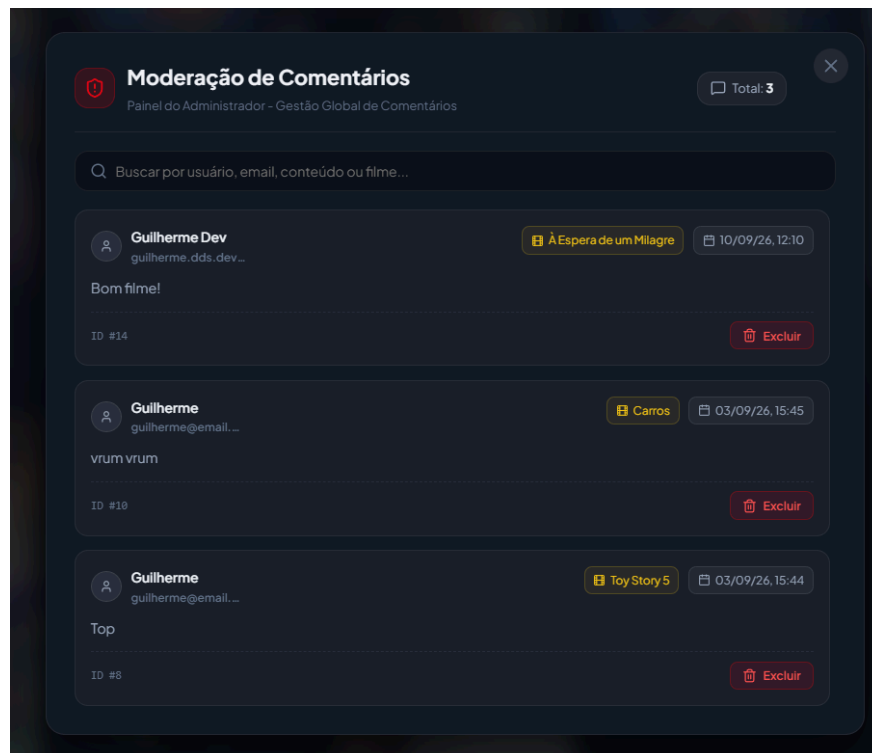


Figura 8: Atividade 4 — resultado

Um problema encontrado foi garantir que as rotas administrativas continuassem protegidas mesmo quando acessadas diretamente pela API, sem passar pela interface. A solução foi adicionar a verificação do role no middleware e nas rotas protegidas, fazendo com que usuários autenticados, mas sem a permissão necessária, recebessem 403 em vez de conseguirem executar a operação.

4.5 Atividade 5 — Logs e auditoria

Descrição	Novo log-service com Redis registrando login, ações sensíveis e tentativas negadas.
Data planejada	25/09/2026

Data realizada	10/09/2026
Situação	entregue
Evidência	GitHub — issue + print da consulta de logs pelo admin
Link	https://github.com/guilherme-dds/movie-catalog

Nesta atividade foi criado um microserviço responsável pelo registro de eventos importantes da aplicação, como login, logout, favoritos, comentários, ações administrativas e tentativas de acesso negadas. O armazenamento dos eventos foi realizado utilizando Redis, mantendo o serviço de logs separado dos demais componentes da aplicação. A decisão de utilizar um microserviço específico permitiu centralizar os registros e evitar que cada serviço precisasse acessar diretamente o armazenamento de logs, enquanto o acesso aos registros foi restringido aos administradores.

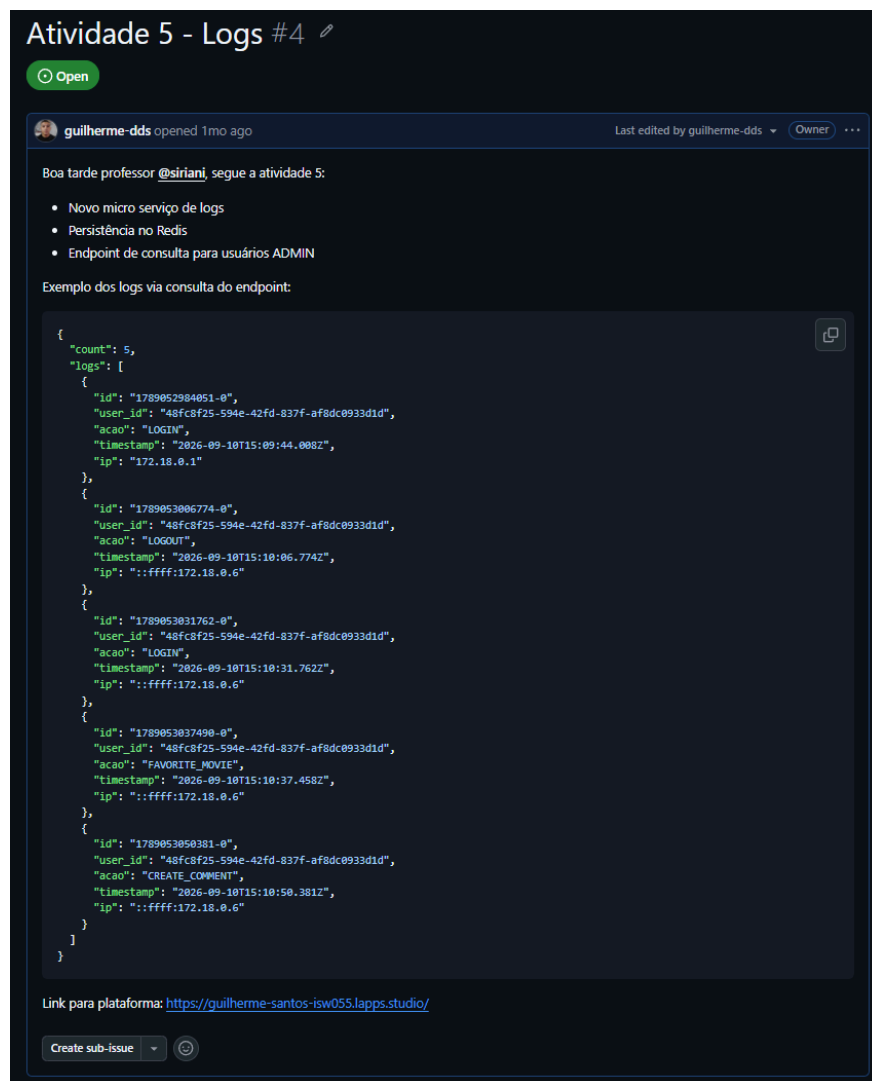


Figura 9: Atividade 5 — evidência da entrega

```

1 {
2   "count": 5,
3   "logs": [
4     {
5       "id": "1791225979680-0",
6       "user_id": "e75524ec-7d2e-4a25-8876-f2b7477a1d20",
7       "acao": "LOGOUT",
8       "timestamp": "2026-10-05T18:46:19.662Z",
9       "ip": "172.18.0.1"
10    },
11    {
12      "id": "1791226668879-0",
13      "user_id": "e75524ec-7d2e-4a25-8876-f2b7477a1d20",
14      "acao": "LOGIN",
15      "timestamp": "2026-10-05T18:57:48.878Z",
16      "ip": "172.18.0.1"
17    },
18    {
19      "id": "1791226749492-0",
20      "user_id": "48fc8f25-594e-42fd-837f-af8dc0933d1d",
21      "acao": "LOGIN",
22      "timestamp": "2026-10-05T18:59:09.492Z",
23      "ip": "172.18.0.1"
24    },
25    {
26      "id": "1791226769552-0",
27      "user_id": "48fc8f25-594e-42fd-837f-af8dc0933d1d",
28      "acao": "DELETE_COMMENT",
29      "timestamp": "2026-10-05T18:59:29.517Z",
30      "ip": "172.18.0.1"
31    },
32    {
33      "id": "1791227286005-0",
34      "user_id": "48fc8f25-594e-42fd-837f-af8dc0933d1d",
35      "acao": "LOGIN",
36      "timestamp": "2026-10-05T19:08:06.005Z",
37      "ip": "172.18.0.1"
38    }
39  ]
40 }

```

Figura 10: Atividade 5 — resultado

A principal dificuldade encontrada foi fazer os diferentes microsserviços enviarem os eventos para o serviço de logs sem depender de um acesso direto ao Redis. A solução foi concentrar essa comunicação no log-service, fazendo os demais serviços enviarem as informações para ele e deixando o acesso ao Redis restrito ao próprio serviço de logs.

4.6 Atividade 6 — Upload e perfil de usuário

Descrição	Página de perfil com avatar no MinIO; só a referência fica no banco relacional.
Data planejada	02/10/2026
Data realizada	01/10/2026
Situação	entregue
Evidência	GitHub — commit + print do perfil com foto
Link	https://github.com/guilherme-dds/movie-catalog

Nesta atividade foi adicionada uma funcionalidade de perfil de usuário, permitindo armazenar informações como nome, biografia e foto de perfil, além dos filmes favoritos. Para o armazenamento das imagens foi utilizado o MinIO como object storage, enquanto o banco de dados ficou responsável apenas pelas informações e referências necessárias aos arquivos. Também foi mantido o controle de acesso implemen-

tado anteriormente para impedir que um usuário alterasse o perfil de outra pessoa por meio de requisições diretas à API.



Figura 11: Atividade 6 — evidência da entrega

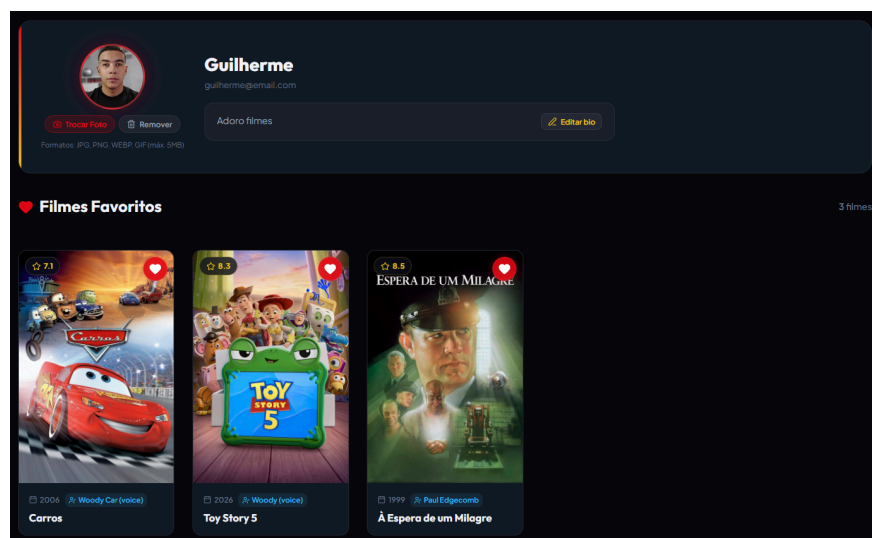


Figura 12: Atividade 6 — resultado

Durante a configuração do ambiente Docker, houve um erro ao tentar baixar a imagem minio/minio:latest, com o Docker retornando que o repositório não existia ou não estava acessível. Para resolver o problema, a imagem utilizada no docker-compose foi alterada para uma imagem compatível com o ambiente, permitindo que o container do MinIO fosse iniciado junto aos demais serviços.

5 Atividades extras (opcional)

5.1 Atividade E1 — Documentação Swagger/OpenAPI

Descrição	Swagger UI com pelo menos 2 serviços documentados e “Try it out” executado.
Data planejada	sem prazo
Data realizada	05/10/2026
Situação	entregue
Evidência	GitHub — issue + print da documentação Swagger
Link	https://github.com/guilherme-dds/movie-catalog

Nesta atividade foi implementada a documentação das APIs utilizando Swagger/OpenAPI, descrevendo as rotas, métodos HTTP, parâmetros, corpos das requisições e possíveis respostas. A documentação foi inicialmente criada para o microserviço de autenticação e posteriormente estendida ao backend principal. Foi mantida a rota /apidocs para acesso à documentação e, como os microserviços utilizam a mesma aplicação publicada externamente, foi utilizado o Nginx para realizar o roteamento das requisições e evitar a necessidade de expor uma nova porta para cada serviço.

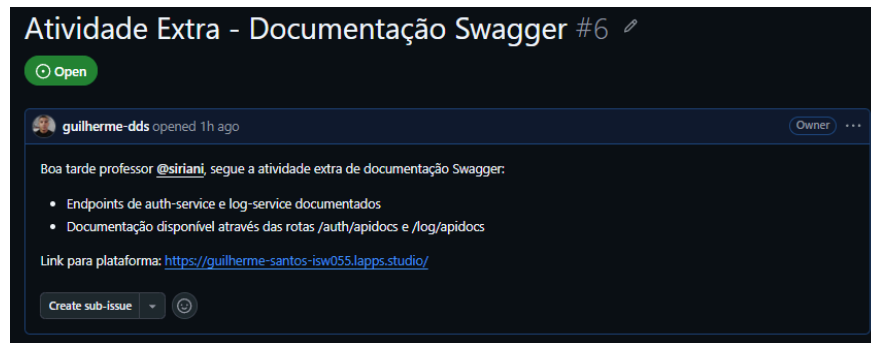


Figura 13: Atividade E1 — evidência da entrega

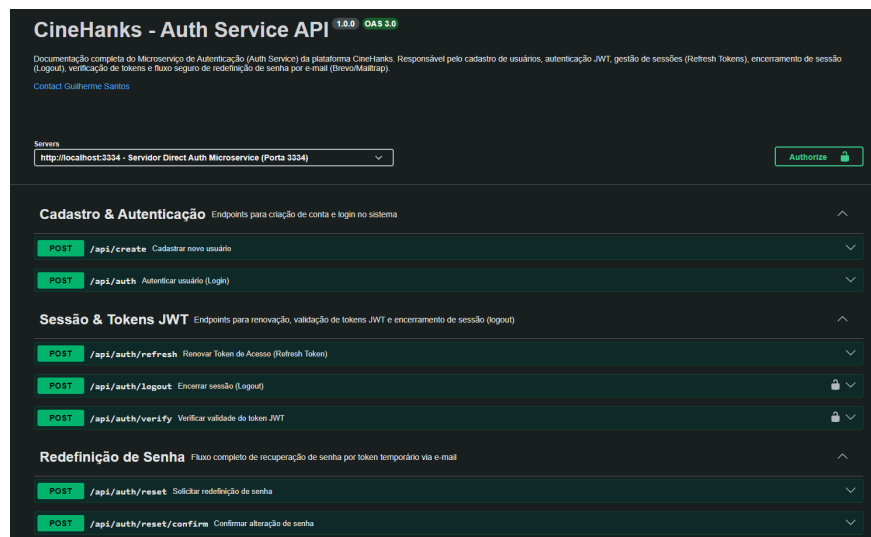


Figura 14: Atividade E1 — resultado)

Uma das dificuldades foi documentar corretamente as rotas que utilizavam autenticação, principalmente aquelas que exigiam o token JWT no cabeçalho da requisição. Foi necessário configurar o esquema de segurança do Swagger para permitir informar o token pelo botão Authorize e testar as requisições diretamente pela interface, verificando se os parâmetros, respostas e códigos HTTP documentados correspondiam ao comportamento real da API.

6 Considerações finais

Ao longo do bimestre, foi possível compreender na prática como diferentes conceitos de computação em nuvem podem ser aplicados durante o desenvolvimento de uma aplicação. A evolução do projeto permitiu aprender não apenas sobre as tecnologias utilizadas, mas também sobre a importância de organizar a aplicação em diferentes serviços, documentar APIs, trabalhar com autenticação e considerar aspectos como persistência, armazenamento e segurança desde o desenvolvimento.

Uma das principais dificuldades foi lidar com a integração entre os diferentes componentes do projeto, especialmente quando uma alteração em um serviço afetava o funcionamento de outro. Esses problemas

ajudaram a desenvolver uma visão mais ampla sobre a comunicação entre serviços e sobre a necessidade de testar cada parte da aplicação durante sua evolução. Em um próximo projeto, seria interessante planejar melhor a arquitetura e a integração entre os serviços desde o início, reduzindo retrabalho e facilitando a identificação de problemas. Os conhecimentos adquiridos neste bimestre servirão como base para aprofundar o uso de arquiteturas distribuídas, serviços em nuvem e boas práticas de desenvolvimento nas próximas atividades.

7 Declaração de autoria

Declaro que este relatório foi elaborado por mim, individualmente, e que as evidências apresentadas correspondem a entregas de minha autoria, verificáveis nos links informados. Nas atividades realizadas em grupo, o conteúdo aqui descrito refere-se à minha participação.



Documento assinado digitalmente

GUILHERME DIAS DOS SANTOS

Data: 07/10/2026 18:02:29-0300

Verifique em <https://validar.iti.gov.br>

Guilherme Dias dos Santos

Pompeia, 07/10/2026